EcoLoop: Technical Implementation and Scope Validation Report

Hephzibah Ofomi | BSc. Software Engineering, African Leadership University | Supervisor: Hubert Apana | October 2026

Repository: https://github.com/H3PHZY/CAPSTONE_PROJECT.git | Walkthrough video: https://www.youtube.com/watch?v=w7FzwgZITX0 | Live app: https://ecoloop-hephzibah1.vercel.app/

1. Project Overview and Architecture

EcoLoop is a web marketplace where SME factories in Lagos's Ogba and Ikeja clusters list surplus industrial material, and nearby buyers find it, negotiate for it and pay for it. A seller uploads a photo, a Vision Transformer (ViT) model suggests one of five material categories (Plastic, Glass, Metal, Biodegradable, Rubber), and the seller can override the suggestion. Buyers filter by material and distance, message the seller, and pay once the seller reserves the lot. Every completed exchange records an estimated CO2e saving.

The app uses Next.js 16 (App Router), React 19 and TypeScript for the pages and API routes, Tailwind CSS 4 for styling, Supabase for PostgreSQL, authentication, file storage and realtime updates, Hugging Face Inference for classification, Sharp for image resizing, and Paystack in test mode for payments. It is deployed on Vercel. Secret keys stay in the API routes in `app/api/`, and row level security (RLS) rules in the database limit what each user can read or change.

2. Scope Alignment and Feature Validation

Planned feature (proposal)	Intended purpose	Status	Evidence
Registration and authentication	Secure access for sellers and buyers	Completed	`lib/auth-context.tsx`, `components/auth/`; Supabase Auth, password reset, company profile with photo
Listing creation with photo upload	List material quickly	Completed	`QuickListingPage.tsx`, `lib/listings.ts`; multiple photos (migration 008). The 60-second target is reported in Section 4.3
ViT classification with manual override	Cut effort and tagging errors	Completed, with a change	`lib/classify.ts`, `app/api/classify/route.ts`; Sharp resize, label mapping to 5 categories, results under 60% confidence flagged
75% accuracy on a Lagos image test set	Show the model works on local waste	Not completed	No evaluation set built yet; planned for the pilot phase

Planned feature (proposal)	Intended purpose	Status	Evidence
Location-aware browsing and proximity filter	Help buyers find nearby material	Completed	`haversineKm` in `lib/listings.ts`, radius options in `lib/data.ts`, `MarketplacePage.tsx`
In-app messaging	Let buyers and sellers negotiate	Completed	`lib/messages.ts`, `MessagesPage.tsx`; realtime updates and file attachments (migrations 011, 012)
Transaction status tracking	Follow a deal from interest to completion	Completed	`lib/transactions.ts`; statuses Interest, Reserved, Completed, Cancelled
Per-transaction carbon estimate	Make the climate benefit visible	Completed, one gap	`lib/carbon.ts`, `carbon_records` table, impact page; the proposal's ±10% margin of error is not yet shown in the UI
Payment gateway (deferred in proposal)	Confirm payment before an exchange completes	Added	`app/api/paystack/`, `components/payments/PayButton.tsx`, migrations 014 and 016; test mode only
Dispute handling, chat encryption, 18+ check	Trust and safety	Not completed	Deferred, see below
Draft listings	Save unfinished listings	Partial	Draft status exists and can be set on the Manage Listing page; no dedicated draft flow
Six-week SME pilot	Test in the real cluster	Not started	Scheduled for November 2026 in the proposal timeline

Scope changes and why

Payments moved into the MVP. The proposal listed payment processing as post-MVP, but my supervisor asked for a payment simulation using a sandbox. Without payment confirmation there is nothing to verify and nothing to stop a lot being claimed twice, so I added Paystack in test mode. This also forced stronger reservation logic, described in Section 5.2.

The model order changed. The proposal names `yangy50/garbage-classification` as the main model with `google/vit-base-patch16-224` as the fallback. In practice the specialist checkpoint often has no free hosted provider, so the code tries the general ViT first, maps its ImageNet labels to the five EcoLoop

categories, and only then tries the specialist model. The label-mapping layer therefore does more work than planned.

Wood and cardboard were folded into Biodegradable (migration 006) so the system keeps exactly five categories, as the proposal states.

Dispute handling, chat encryption and the 18+ check were deferred because of the compressed timeline. Their risks are reduced for now by RLS on every table, server-side payment verification and the reservation rules, and they are listed as future work.

3. Technical Functionality

3.1 AI classification pipeline

The seller's photo is stored in Supabase Storage. The browser then calls `/api/classify` with the photo URL. The route checks that the URL belongs to the project's own storage, then `classifyListingImage` downloads the image, rotates and resizes it to at most 768 px with Sharp, and sends it to Hugging Face. If one call style fails, it tries another (binary upload, base64 body, SDK), and then the next model. The raw labels are cleaned and matched to a category by a lookup table and whole-word regular expressions, so a label like 'water bottle' becomes Plastic and 'tyre' becomes Rubber. The result returns the category, confidence and alternatives. Anything under 60% confidence is flagged for review. The listing stores the prediction, confidence, model used and whether the seller overrode it in the `classifications` table.

3.2 Marketplace and location filtering

Listings carry coordinates taken from the industrial zone the seller picks (Ogba, Ikeja, Isolo, Apapa), and the viewer's position comes from their cluster. The distance between them is calculated with the haversine formula and used for the radius filter (5, 10, 25 km or all Lagos), together with material, search text, verified-seller and sort options. Because coordinates are zone centres and not GPS points, distances are approximate, which is enough for cluster-level matching.

3.3 Messaging, reservation and payment flow

A buyer clicks Initiate contact, which opens a thread with the seller. Messages update live through Supabase Realtime and support attachments. The seller reserves the lot for one buyer (`reserve_listing_for_buyer`) and sets the price. The buyer then pays: `/api/paystack/initialize` checks the buyer's login, issues a server-made reference in the form `ECO-<listing>-<timestamp>-<random>` and the amount in kobo from the database. The Paystack popup uses only those values. Afterwards `/api/paystack/verify` asks Paystack to confirm the payment, checks the reference, currency and amount against the agreed price, and calls `confirm_paystack_payment` to record it. The exchange can only be marked Completed once payment is confirmed, and completion sets the listing to Sold.

3.4 Carbon impact

When an exchange completes, the weight is multiplied by a CO2e factor for the material (for example Metal 4.2, Plastic 1.9 kg CO2e per kg) and saved in `carbon_records`. The dashboard and impact page add these up. The figures are labelled as estimates, not certified offsets.

4. Testing, Performance and Compatibility

4.1 Test 1: Integration and end-to-end payment verification

Why: a payment touches the browser, my API routes, Paystack and the database, so unit tests alone cannot show these parts agree with each other or resist tampering.

How: three accounts (Seller, Buyer A, Buyer B) on separate browsers, running locally against Supabase and Paystack test mode. I reserved a lot for Buyer A, watched the requests in the browser Network tab, paid with Paystack test card 4084 0840 8408 4081, and checked the database with SQL before and after each step. [TO FILL: test date]

Case	Expected result	Actual result
Click Pay	`initialize` payload holds only the transaction id; response has an `ECO-` reference and the amount in kobo	[TO FILL]
Call `initialize` without a token	401 'Please log in again'	[TO FILL]
Call `initialize` with an unknown transaction id	404 'Deal not found'	[TO FILL]
Call `verify` with a made-up reference	400 'Payment reference was not issued for this deal'	[TO FILL]
Pay with the test card	`verify` returns 200; `paid_at` and `payment_ref` are set; listing still Reserved	[TO FILL]
Re-send the same `verify` request	409 'This deal is already paid'	[TO FILL]
Mark completed	Transaction Completed, listing Sold, one row in `carbon_records`	[TO FILL]

4.2 Test 2: Concurrency and lot locking

Why: two buyers must never be able to claim the same lot, which was a specific concern raised by my supervisor.

How: first by hand, with the Seller reserving Buyer A and Buyer B from two tabs back to back. Then with a Node script (`tests/concurrency/reserve-race.mjs`) that sends both reservation calls at the same instant with `Promise.all`, repeated [TO FILL: number] times, resetting the lot between runs. The database function locks the listing row (`FOR UPDATE`) so the second call waits, sees the first reservation and is rejected.

Case	Expected result	Actual result
Manual two-tab reservation	First succeeds; second shows 'This listing is already reserved for another buyer'	[TO FILL]
Scripted simultaneous reservation, repeated	Exactly one RESERVED and one REJECTED every run	[TO FILL]
Check transactions for the lot afterwards	Only one buyer has a Reserved row	[TO FILL]
Double-click Pay	One `initialize` request; the button is disabled while it runs	[TO FILL]

4.3 Performance and compatibility

Environment or metric	Result	Notes
Time to create a listing (target 60 s)	[TO FILL]	[TO FILL: how many people, how timed]
Lighthouse performance (live app)	[TO FILL]	[TO FILL: desktop or mobile]
First Contentful Paint	[TO FILL]	
Chrome desktop	[TO FILL]	
Firefox or Edge desktop	[TO FILL]	
Android phone browser	[TO FILL]	[TO FILL: device model]

4.4 Improvements after testing

[TO FILL: list only changes made because a test failed. If no defect was found, write that and name one hardening step, for example adding a Paystack webhook so payment is still recorded if the buyer closes the browser.]

5. Code Quality, OOP Principles and Algorithms

5.1 Structure and separation of concerns

The repository is split by responsibility. `app/` holds routes and API endpoints, `components/` holds the interface grouped by feature (auth, marketplace, listings, messages, payments, profile), `lib/` holds data access and business logic one concern per file (`listings.ts`, `transactions.ts`, `messages.ts`, `classify.ts`, `carbon.ts`, `payment-pricing.ts`), and `supabase/migrations/` holds 16 numbered SQL files that rebuild the database. Secrets and payment logic stay on the server, and the rules that must never break (one reserved buyer per lot, no completion without payment) are enforced in the database itself, not only in the interface. The code is written as TypeScript modules and React components, not heavy class hierarchies, which suits this stack. Types in `lib/types.ts` and `lib/database.types.ts` keep data shapes consistent, and navigation links are defined once in `lib/navigation.ts`.

5.2 Algorithms and problem solving

- Haversine distance for proximity filtering (`lib/listings.ts`).
- Label normalisation: exact lookup, then whole-word regular expressions, then ImageNet object rules ordered so that Rubber is checked before Metal, followed by a confidence threshold that flags uncertain results.
- A fallback chain across request styles and models so classification still works when a provider is unavailable.
- Atomic reservation and payment in PostgreSQL (migration 016): row locks with `FOR UPDATE`, a trigger that blocks a second reserved buyer, a server-issued payment reference checked against a pattern and a 30 minute expiry, and an amount check in kobo on both the API and the database.

5.3 Known limitations and planned refactoring

Some components are still large, notably `MessagesPage.tsx` at about 1,000 lines, and `lib/classify.ts` repeats similar request code for two call styles. My next step is to split these into smaller components and hooks, and to put the classifiers behind one interface so a new model can be added without editing the fallback logic. The project also has no automated unit tests yet, so testing so far relies on the two strategies above.

6. Conclusion

EcoLoop implements the full proposed MVP: authentication, AI-assisted listing with override, proximity search, messaging, transaction tracking and carbon estimates. It also adds sandbox payments with database-level protection against double claims. The main gaps are the local accuracy evaluation, pilot analytics and the deferred trust features, all planned for the pilot phase.